

Lab 15 – Manipuler des VPC

Dans cet atelier, on va montrer comment utiliser VPC avec Terraform.

Configuration du VPC

Etape 1: Créer le VPC

Dans un nouveau répertoire, créer un fichier `vpc.tf`. Pour créer un VPC, il s'agit de créer une ressource "aws_vpc" avec le nom "main".

```
# Internet VPC
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  instance_tenancy = "default"
  enable_dns_support = "true"
  enable_dns_hostnames = "true"
  enable_classiclink = "false"
  tags = {
    Name = "main"
  }
}
```

- Et, également dans AWS, nous lui attribuons un tag de nom **Name** à "main". Ainsi, vous pouvez voir plus tard dans l'interface utilisateur que vous avez un VPC appelé "main".
- Une plage d'adresses privées **cidr_block = "10.0.0.0/16"**, par la suite dans ce VPC on créera des sous-réseaux de **"/24"**.
- **instance_tenancy = 'default'**, signifie que chaque instance que vous lancez aura une location par défaut. Ainsi, vous pouvez avoir plusieurs instances sur un même matériel physique. Vous pouvez changer cela si vous avez besoin de n'avoir qu'une seule instance par machine physique, mais comme cela coûte beaucoup plus cher, la plupart des gens la laissent par défaut.
- **enable_dns_support = "true"** et **enable_dns_hostnames = "true"** : pour activer la prise en charge DNS et les noms d'hôte DNS, qui vous fournissent un nom d'hôte et un nom de domaine internes sur votre instance.
- **enable_classiclink = "false"** : si vous souhaitez lier votre VPC à EC2 classic, vous pouvez le définir comme "true". Si vous n'avez rien dans EC2 Classic, alors vous le gardez sur "false".

Etape 2: Créer les sous-réseaux (Subnets).

Nous allons avoir 3 sous-réseaux privés et 3 publics.

Nous commençons avec les 3 sous-réseaux publics, ajoutez le code suivant juste au dessous du VPC:

```

# Subnets
resource "aws_subnet" "main-public-1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  map_public_ip_on_launch = "true"
  availability_zone  = "eu-west-1a"

  tags = {
    Name = "main-public-1"
  }
}

resource "aws_subnet" "main-public-2" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  map_public_ip_on_launch = "true"
  availability_zone  = "eu-west-1b"

  tags = {
    Name = "main-public-2"
  }
}

resource "aws_subnet" "main-public-3" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.3.0/24"
  map_public_ip_on_launch = "true"
  availability_zone  = "eu-west-1c"

  tags = {
    Name = "main-public-3"
  }
}

```

Nous nous référons au VPC "**aws_vpc.main**", avec son ID,

cidr_block = 10.0.1.0/24 : ce n'est que 256 adresses IP, moins les adresses réservées que nous ne pouvons pas utiliser.

map_public_ip_on_launch = "true": Vous allez attribuer à chaque instance une adresse IP publique au lancement. Les instances obtiendront une adresse IP de cette plage cidr et également une adresse IP publique pour se connecter à Internet.

availability_zone = "eu-west-1c": La "zone de disponibilité" dans ce cas va être, "eu-west-1a", si vous voulez lancer dans une autre région vous voudrez peut-être changer cela.

Et puis nous avons juste la même chose pour les deux autres sous-réseaux.

Ajoutez à la suite les trois sous-réseaux privés comme suit:

```

resource "aws_subnet" "main-private-1" {
  vpc_id            = aws_vpc.main.id

```

```

cidr_block      = "10.0.4.0/24"
map_public_ip_on_launch = "false"
availability_zone = "eu-west-1a"

tags = {
  Name = "main-private-1"
}
}

resource "aws_subnet" "main-private-2" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = "10.0.5.0/24"
  map_public_ip_on_launch = "false"
  availability_zone = "eu-west-1b"

  tags = {
    Name = "main-private-2"
  }
}

resource "aws_subnet" "main-private-3" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = "10.0.6.0/24"
  map_public_ip_on_launch = "false"
  availability_zone = "eu-west-1c"

  tags = {
    Name = "main-private-3"
  }
}

```

La principale différence est que nous ne mappons pas une adresse IP publique lors du lancement pour les sous-réseaux privés.

Etape 3: Créer une passerelle Internet Gateway

nous avons besoin de définir une passerelle Internet, car les sous-réseaux publics doivent être connectés à une passerelle pour se connecter à Internet.

Ajoutez le code suivant à la suite:

```

# Internet GW
resource "aws_internet_gateway" "main-gw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "main"
  }
}

```

Cette ressource crée une passerelle Internet dans le VPC déjà créé.

Etape 4: Contrôler les communication avec une table de routage AWS

Nous allons pousser une table de routes aux instances, dans laquelle nous définissons une route pour acheminer tout le trafic non interne vers toutes les adresses IP "0.0.0.0/0", sauf celles qui correspondent à la plage du VPC, via la passerelle Internet.

```
# route tables
resource "aws_route_table" "main-public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main-gw.id
  }

  tags = {
    Name = "main-public-1"
  }
}
```

Dans ce cas, si vous souhaitez contacter un site Web, ce site Web aura une adresse IP différents de votre VPC. Donc, cela passera par la passerelle Internet.

Il reste d'associer la table de routage au sous réseaux publics.

Etape 5: Associer la table de routage aux sous-réseaux

Les règles de routage ne seront considérées par les instances, que si nous associons cette table de routage déjà créée pour chaque sous-réseau public.

Vous avez 3 sous-réseaux publics, il faut donc créer une ressource **aws_route_table_association** pour chaque sous-réseau:

```
# route associations public
resource "aws_route_table_association" "main-public-1-a" {
  subnet_id    = aws_subnet.main-public-1.id
  route_table_id = aws_route_table.main-public.id
}

resource "aws_route_table_association" "main-public-2-a" {
  subnet_id    = aws_subnet.main-public-2.id
  route_table_id = aws_route_table.main-public.id
}

resource "aws_route_table_association" "main-public-3-a" {
  subnet_id    = aws_subnet.main-public-3.id
  route_table_id = aws_route_table.main-public.id
}
```

C'est tout pour le VPC et les sous-réseaux publics.

Lab : Créer une Instance EC2 dans un VPC

Dans cette courte démonstration, vous allez lancer une instance AWS avec un groupe de sécurité, dans un VPC, avec une paire de clés.

En se basant sur le Lab de VPC, si vous avez déjà créé votre VPC, vous voudrez peut-être copier votre "terraform.tfstate" et votre "terraform.tfstate.backup".

Sinon, vous pouvez simplement recréer le VPC. Si vous obtenez une erreur, vous devez d'abord le supprimer.

Etape 1: préparer les clés RSA pour la connexion SSH

Créez la paire de clés RSA "mykey" et "mykey.pub", rappelez-vous que nous pouvons le faire en utilisant

```
ssh-keygen -f mykey
```

Etape 2: Créer la configuration de l'instance EC2

Regardons "instance.tf".

```
resource "aws_instance" "example" {
  ami           = var.AMIS[var.AWS_REGION]
  instance_type = "t2.micro"

  # the VPC subnet
  subnet_id = aws_subnet.main-public-1.id

  # the security group
  vpc_security_group_ids = [aws_security_group.allow-ssh.id]

  # the public SSH key
  key_name = aws_key_pair.mykeypair.key_name
}
```

subnet_id = aws_subnet.main-public-1.id : sous-réseau à lancer, nous obtenons donc une adresse IP de main-public-1, et si on regarde dans **vpc.tf**, le main-public-1 signifie qu'il obtiendra une adresse IP de la plage **10.0.1.0/24** et il la lancera "eu-west-1a"

Etape 3: planifier et appliquer l'approvisionnement

Essayons d'abord, terraform plan, juste pour voir ce qui se passe avant d'appliquer.

Vérifiez: - Le VPC déjà créé - Les différentes ressources à créer

Nous allons maintenant lancer cette instance AWS "T2.micro", avec "mykeypair", dans ce sous-réseau et nous allons autoriser le port 22 "ssh".

Donc, tapez simplement `terraform apply`.

Etape 4: Tester la configuration

1. Essayez de vous connecter à l'instance EC2 avec SSH: En cas de besoin, modifier le code et ajouter un champ de sortie "output" pour afficher l'adresse IP.

Ou ce que vous pourrez aussi faire est d'aller rapidement sur "terraform.tfstate", ouvrez-le. Recherchez l'adresse IP publics.

Alors, connectons-nous à cette adresse IP en utilisant ubuntu comme identifiant et cette "mykey", la clé privée, en tant que clé.

```
shell ssh <INSTANCE_IP> -l ubuntu -i mykey
```

2. Vérifier l'adresse IP privé de l'instance, qu'elle soit dans la plage de `main-public-1`

```
shell sudo -i ip a s
```

cherchez l'interface `eth0`, et vérifiez que l'adresse IP interne dans la plage de sous-réseau "10.0.1.XX". Vous pouvez également voir qu'il existe un masque sous-réseau sur "/24".

3. Vérifier les routes disponible dans la table de routage:

```
shell route -n
```